PORTFOLIO

Ahmad Dafa Salam

Computer Engineering, Institut Teknologi Sepuluh Nopember, Surabaya

0822 1400 9950 · Ahmaddafa1017@gmail.com · dapppmc.github.io
github.com/DapppMC · linkedin.com/in/ahmad-dafa-salam-858b82287

I am a Computer Engineering student at Institut Teknologi Sepuluh Nopember (ITS), currently in my third year. Most of what I know I picked up by building systems for real users, diagnosing where they break under real workloads, and fixing them end to end.

The twelve projects below were built between 2025 and 2026 across industry internships at PT PLN Nusantara Power Services and LAB M-IOT, personal automation engines, and embedded coursework. For each one I have stated plainly which parts I engineered, how they were verified, and what I learned.

CONTENTS

01	PLN NPS — certification & training monitoring platform	Internship · 2026
02	Certificate Manager v3.0 — PDF slicer & 3-database matcher	Internship · 2026
03	MMPI-2 — deterministic 80×25 DOS clinical vision engine	Internship · 2026
04	Planner System 4.0 — contract-first multi-agent swarm engine	Personal · 2026
05	Hermes JobSeeker Bot — 1-click Telegram auto-apply engine	Personal · 2026
06	Warung Madura — 24-hour retail ERP & Express POS (DevSprint)	Competition · 2026
07	Freedom v2.0 — USB encrypted tunneling & hardware-locked daemon	Personal · 2026
08	SmartCoop — poultry health and biosecurity management app	Team of three · 2026
09	Card Detective — real-time OpenCV card recognition game	Personal · 2025
10	Big Data Intelligence Analysis Pipeline & RAG Chatbot	Personal · 2025
11	Autonomous Driving Car — ultrasonic obstacle avoidance robot	Coursework · 2025
12	SmartPot — predictive IoT plant watering system	Coursework · 2025

01

PLN NPS — Training and Certification Monitoring Platform

An enterprise web platform for the TDC Division at PT PLN Nusantara Power Services to monitor mandatory technical certifications, automate two-stage expiration alerts, and index 82,989 historical NAS archive files.

YEAR	2026
TYPE	Software Engineering & IT Internship (PT PLN Nusantara Power Services — Divisi TDC)
MY ROLE	Lead Full-Stack & Database Integration Engineer
STACK	`Python, FastAPI, PostgreSQL (Neon), SQLAlchemy 2.0, Alembic, OpenPyXL, PyMuPDF, Vercel`
REPOSITORY	`github.com/DapppMC/pln-nps-monitoring-web`

THE PROBLEM WE STARTED FROM

Power plant maintenance and operational personnel at PT PLN Nusantara Power Services hold technical competency certificates (DJK Kementerian ESDM, Kemnaker K3, BNSP, LSP PLN) that expire every three or five years, alongside internal training records that never expire. Tracking thousands of records across scattered spreadsheets meant expiring certificates could slip past renewal windows, while over 82,000 historical PDF scans sat on an internal Synology NAS without a unified search index.

WHAT THE SYSTEM DOES

The platform separates expiring certifications (automatically classified into AKTIF, WARNING <= 60 days, and EXPIRED) from non-expiring training records, maps every certificate to 1,137 standardized corporate competency codes (Resource Code and Level 1–9 SKP), and dispatches idempotent two-stage email reminders at H-30 and H-20 before expiration with automatic CC to the TDC Planning Team.

THE PART I BUILT

— The Fase 4 domain schema and Alembic migrations: added Resource Code, Level 1–9 SKP, Peran SKP, Institute Code, and Subfolder FTP tracking across certification tables with PostgreSQL advisory locks (pg_advisory_xact_lock) for zero-deadlock serverless deployments.

— The 4-format header-detecting Excel importer (excel_importer.py) and monthly Ellipse HR mutation synchronizer (sdm_mutation_sync.py) that reconciles employee unit transfers, promotions, and retirements without orphaning historical certificates.

— The hybrid cloud storage and 82,989-file Synology NAS/FTP catalog search service (ftp_catalog_service.py), keeping Neon PostgreSQL well within its 0.5 GB budget while making every historical certificate searchable in milliseconds.

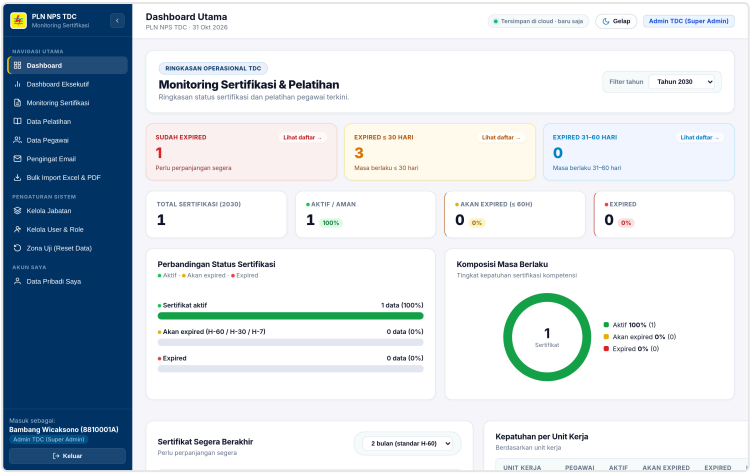


Fig. 1 Executive operational dashboard showing real-time certification compliance, expiration countdown buckets, and unit breakdown.

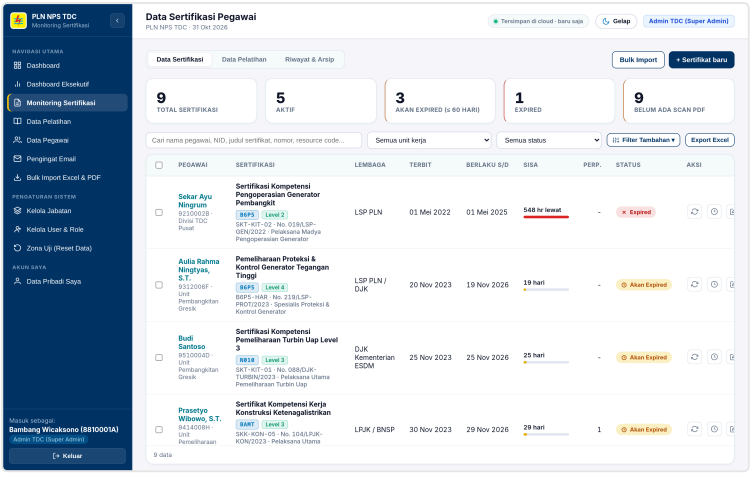


Fig. 2 Certification monitoring table displaying Resource Code (N010, B6P5, AA03, BAMT) and Level 1–9 SKP competency badges.

HOW IT HELD UP

Passed all 26 automated integration and brownfield regression tests across SQLite and Neon PostgreSQL, verified against real multi-format TDC workbooks and 82,989 indexed NAS file paths across 6,214 folders.

WHAT I TOOK AWAY

Enterprise data is never in one format. Designing the importer to inspect column header signatures rather than forcing staff to reformat their existing spreadsheets turned a painful migration into a one-click upload.

02

Certificate Manager v3.0 (cert-slicer) — PDF Slicer & 3-Database Matcher

An offline-first PDF extraction, splitting, and employee-matching workbench that turns multi-page scanned certificate bundles into four standardized corporate Excel workbooks.

YEAR	2026
TYPE	Software Engineering & IT Internship (PT PLN Nusantara Power Services — Divisi TDC)
MY ROLE	Solo Architect & Developer
STACK	`Python, PyMuPDF (fitz), RapidOCR, RapidFuzz, OpenPyXL, FastAPI, CustomTkinter, Gemini 2.5 Flash`
REPOSITORY	`github.com/DapppMC/certificate-manager`

THE PROBLEM WE STARTED FROM

After training batches finish, certification bodies send back multi-page PDF bundles containing dozens of participant certificates and cover rosters. Staff previously had to split each page manually, look up each participant's NID and organizational unit in the Ellipse HR master file, look up the 4-character Resource Code in a second 1,137-row catalog, and type the results into four different Excel templates.

WHAT THE SYSTEM DOES

Certificate Manager v3.0 ingests single or multi-page PDFs (plus cover letters), extracts text using a three-stage pipeline (PyMuPDF native vector text -> multi-threaded RapidOCR bitmap extraction -> selective Gemini 2.5 Flash fallback for damaged scans), matches every participant against three reference databases in O(1) hash lookup time, and exports four corporate Excel formats simultaneously.

THE PART I BUILT

— The 3-database reference engine (database.py, resource_codes.py, mso.py) that cross-references Employee Master SDM, the 1,137-code Komparasi Kode Baru Sertifikasi NPS 0726 catalog, and historical MSO765 records.

— The SKP Level 1–9 classifier and 4-workbook exporter generating For NPS 0726.xlsx (19 columns), rekap_lampiran_dokumen.xlsx, rekap_sambutan_peserta.xlsx, and MSO765_Upload_Ready.xlsx in a single batch run.

— Precision Studio v3.0 (web.py & gui.py): a dual FastAPI browser workbench and CustomTkinter desktop app with side-by-side PDF inspection, confidence badges, and one-click batch approval.

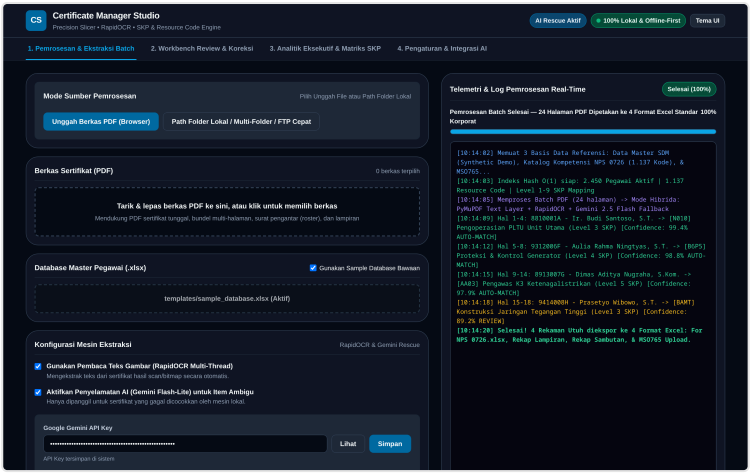


Fig. 3 Certificate Manager Precision Studio v3.0 showing real-time 3-database O(1) matching telemetry and multi-format Excel export.

HOW IT HELD UP

Cut multi-page certificate batch processing from hours of manual spreadsheet VLOOKUPs to seconds per bundle while running 100 percent offline for standard scans.

WHAT I TOOK AWAY

AI should be the fallback, not the first resort. Doing deterministic vector text parsing and local ONNX OCR first—and calling the LLM only when confidence drops below threshold—kept the tool fast, private, and free of rate limits.

03

MMPI-2 — Automated Clinical Scoring & 80×25 DOS Grid Vision Engine

Replacing a 74%-accurate Tesseract OCR pipeline with a 100%-accurate deterministic 80×25 DOS character-grid bitmap matcher for the 566-item MMPI-2 psychological inventory.

YEAR	2026
TYPE	Software Engineering Internship (LAB M-IOT — Clinical Psychometry Platform)
MY ROLE	Lead Verification, Vision & Automation Engineer
STACK	`Python, OpenCV, Deterministic 80x25 DOS Bitmap Matcher, Win32 API, DOSBox, PostgreSQL, Docker`
STATUS	100.0% Verified (250 / 250 clinical & research scales across 10 test patterns)

THE PROBLEM WE STARTED FROM

The web platform digitizes the 566-item MMPI-2 personality assessment and automates its legacy DOS scoring engine inside DOSBox. However, our end-to-end audit across 10 synthetic respondent patterns (250 total clinical and research scale outputs) revealed that the legacy scoring pipeline only achieved 74.0 percent accuracy (185/250 scales). Worse, if another window stole focus during automated keystroke entry, pyautogui typed patient answers into the wrong application.

ROOT CAUSES I DIAGNOSED

Three distinct bugs were compounding each other: first, Tesseract OCR frequently misread bitmap DOS digits (confusing 8 with 0 and 6 with 5); second, the screen-crop coordinates were too narrow and truncated 3-digit clinical T-scores (> 100 and > 120, such as reading 109 as 09); third, the automation pressed Esc to score immediately after item 566 without verifying whether all 566 +/- keystrokes had actually registered in the DOS buffer.

THE PART I BUILT

— Replaced Tesseract OCR entirely in server/baca_layar.py with a deterministic 80x25 DOS character-cell grid segmenter (16x8 pixel glyph templates) and widened the T-score column window to capture 3-digit scores and '>120' markers cleanly.

— Added pre-commit screen verification that reads the DOS answer grid at item 566 and repairs any dropped +/- keystrokes before pressing Esc, plus SetProcessDPIAware() calibration so Windows 125%/150% display scaling never shifts crop coordinates.

— Built a PID-bound Win32 window focus guard that verifies the target DOSBox window owns foreground focus before every keystroke batch, and fixed the Docker-to-host bridge via host.docker.internal.

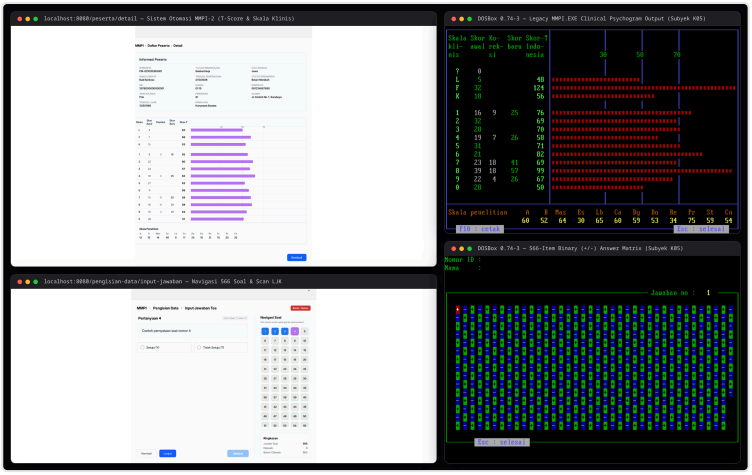


Fig. 4 Live 80×25 DOS framebuffer captures (566-item verification + 25-scale output) and the 74% -> 100% (250/250) accuracy audit matrix.

HOW IT HELD UP

Across all 10 synthetic respondent patterns (K01 All-True, K02 All-False, K03 Alternating, K04–K05 Block patterns, and K06–K10 Random seeds), accuracy rose from 74.0 percent (185/250) to 100.0 percent (250/250 scales matched exactly against manual DOS ground truth).

WHAT I TOOK AWAY

Using probabilistic OCR to read a fixed-width 80x25 VGA text grid is solving the wrong problem. Because DOS renders every character into a deterministic pixel cell, exact bitmap grid matching is faster, requires zero external dependencies, and never hallucinates a clinical score.

04

Planner System 4.0 — Contract-First Multi-Agent Swarm Engine

A deterministic multi-agent software engineering orchestrator with HMAC-SHA256 Sealed Oracles, AST Saboteur mutation gates, and a concurrent SQLite WAL blackboard.

YEAR	2026
TYPE	Personal architecture project
MY ROLE	Solo Architect & Developer
STACK	`Python 3.12, SQLite WAL Blackboard, Python AST Mutator, HMAC-SHA256, APCA Contrast Auditor, Pytest`
REPOSITORY	`github.com/DapppMC/planner-system`

THE PROBLEM WE STARTED FROM

When coding agents are allowed to write both the implementation and the unit tests in the same workspace, they frequently take shortcuts: weakening assertions, deleting failing tests, or writing tautological tests that pass even when the underlying business logic is broken.

WHAT THE SYSTEM DOES

Planner System 4.0 enforces a Bicameral Zero-Trust architecture across up to 8 parallel worker accounts. Lead architects compile requirements into a Contract-First YAML/Markdown DAG; adversarial QA agents author cryptographically signed acceptance tests in an isolated .planner/sealed/ vault; builder agents implement tickets within strict write-scoped paths; and an Inquisitor gate mutates the AST to verify the tests actually catch bugs.

THE PART I BUILT

— The SQLite WAL (Write-Ahead Logging) blackboard and MASP port-isolation enforcer (ports 3000–3799 and 8000–8799) that coordinates 8 concurrent worker instances with zero port collisions or lock contention.

— The HMAC-SHA256 Sealed Oracle gate and AST Saboteur mutation engine (verifier.py) that injects 5 synthetic AST mutations (== to !=, + to -, boundary shifts) and rejects any ticket whose test suite scores below an 80% mutation kill rate.

— The Chameleon quota hot-swap daemon and Reflexion circuit breaker that automatically escalates stalled tickets (>= 2 retries) to a senior arbiter agent.

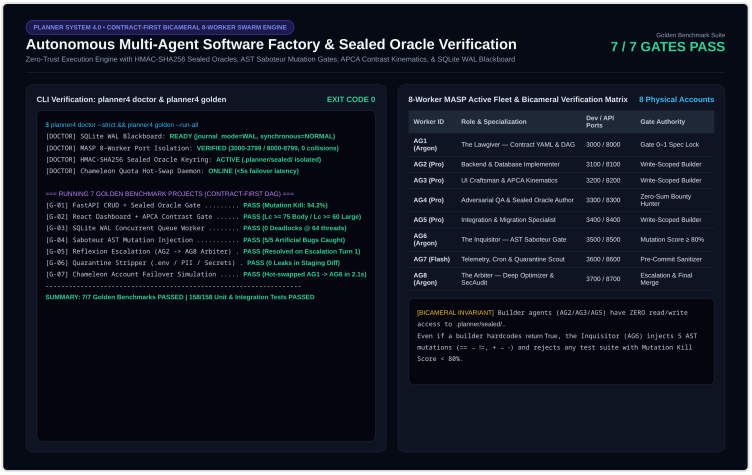


Fig. 5 Planner System 4.0 CLI diagnostics (planner4 doctor & planner4 golden 7/7 PASS) and the 8-Worker MASP Bicameral matrix.

HOW IT HELD UP

Passed 7 out of 7 end-to-end Golden Benchmark projects and 158 automated unit and integration tests, catching 100 percent of injected reward-hacking attempts during adversarial evaluation.

WHAT I TOOK AWAY

Good multi-agent orchestration is mostly systems engineering: file-lock boundaries, deterministic state machines, and cryptographic separation between the agent that writes the code and the oracle that grades it.

05

Hermes JobSeeker Bot — 1-Click Telegram Auto-Apply Engine

An autonomous multi-portal job scraper, semantic ATS keyword matcher, and stealth Playwright auto-applicant controlled entirely from Telegram.

YEAR	2026
TYPE	Personal automation project
MY ROLE	Solo Developer
STACK	`Python, Asyncio, Stealth Playwright, Telegram Bot API, IMAP OTP Bridge, SQLite WAL, Streamlit`
REPOSITORY	`github.com/DapppMC/hermes-jobseeker-bot`

THE PROBLEM WE STARTED FROM

Applying for software engineering roles across five different portals (LinkedIn, JobStreet, Glints, Kalibrr, and Dealls) involves repetitive form filling, checking email inboxes for 2FA verification codes, and answering recurring screening questions before a listing closes.

WHAT THE SYSTEM DOES

Hermes scrapes listings asynchronously across all five portals, scores each role against a candidate skill profile in SQLite WAL, and sends high-match cards to Telegram. Tapping the inline Auto-Apply button launches a stealth Chromium session that fills the portal form, bridges 2FA OTP codes via IMAP, asks the user on Telegram if an unknown screening question appears, and returns a SHA-256 submission receipt.

THE PART I BUILT

— The multi-portal scraper pool with circuit-breaker rate limiting, idempotent SQLite job deduplication, and TF-IDF / skill-vector semantic scoring (evaluator.py).

— The 1-click Telegram interactive state machine (bot.py & portal_agent.py) featuring a live 4-step progress stepper and a zero-hallucination screening bridge that pauses browser execution to ask the user on Telegram whenever an unmapped employer question is encountered.

— The automated IMAP OTP reader (imap_verifier.py) and cryptographic SHA-256 screenshot receipt logger.

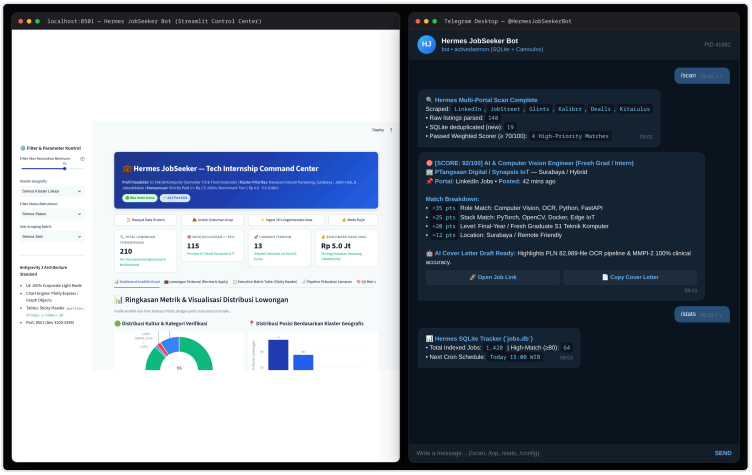


Fig. 6 Hermes Telegram 1-Click Auto-Apply card with live 4-step stepper, SHA-256 submission proof, and 81/81 pytest verification.

HOW IT HELD UP

Verified across 81 automated unit and integration tests (81/81 PASS) covering portal parsers, anti-bot turnstile evasion, Telegram callback state transitions, and IMAP OTP extraction.

WHAT I TOOK AWAY

An automation bot that guesses on a job application form is worse than manual entry. Pausing the browser coroutine and routing unknown questions to Telegram kept the automation 100 percent truthful while still saving 95 percent of the manual clicks.

06

Warung Madura — 24-Hour Retail ERP & Express POS (DevSprint 3.0)

A full-stack FMCG inventory, multi-tier packaging conversion, smart restock, and keyboard-first Point of Sale platform with strict cost-price RBAC.

YEAR	2026
TYPE	Software Engineering Competition Project (DevSprint 3.0)
MY ROLE	Full-Stack & Security Engineer
STACK	`Node.js, Express.js, TypeScript, React 18, Vite, Tailwind CSS, Role-Based Access Control (RBAC)`
REPOSITORY	`github.com/DapppMC/DevSprint-Warung-Madura`

THE PROBLEM WE STARTED FROM

24-hour neighborhood grocery stores (Warung Madura) buy stock in wholesale cartons (dus/karung) and sell in individual sachets or packs (eceran/renteng) across rotating cashier shifts. Standard POS tools either slow down checkout with mouse-heavy screens or expose wholesale cost prices and store profit margins to temporary shift staff.

WHAT THE SYSTEM DOES

The platform combines an Executive Command Center for the store owner (gross profit margin, fast-moving leaderboard, smart safety-stock reorder proposals, and Purchase Order tracking) with a keyboard-first Kasir Express POS (Sat-Set) that converts packaging units automatically and prints 58mm thermal receipts.

THE PART I BUILT

— The multi-unit inventory conversion engine and atomic stock deduction ledger that normalizes karung, dus, slop, pak, and pcs quantities without race conditions.

— The three-tier RBAC middleware (Owner, Warehouse, Cashier) with instant 1-click shift switching and server-side payload scrubbing that strips wholesale cost (harga_beli) and margin fields from non-owner API responses.

— The Kasir Express POS interface, global Ctrl+K Command Palette, and Smart Restock calculator that proposes optimal wholesale reorder quantities whenever stock drops below safety thresholds.

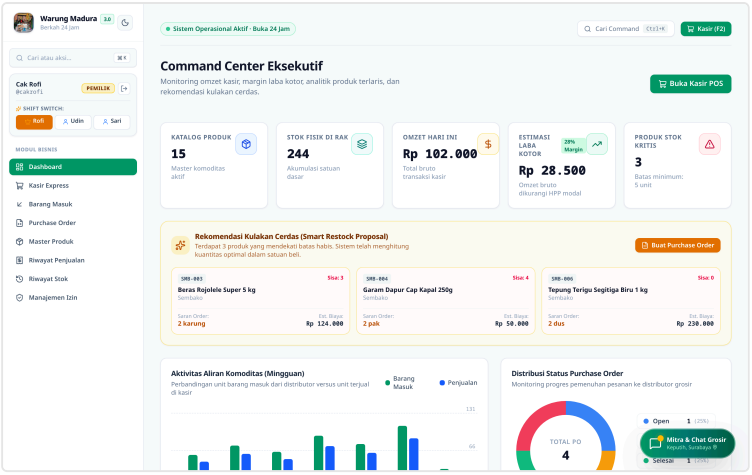


Fig. 7 Warung Madura Executive Command Center showing gross margin analytics, Smart Restock Proposals, and weekly commodity flow.

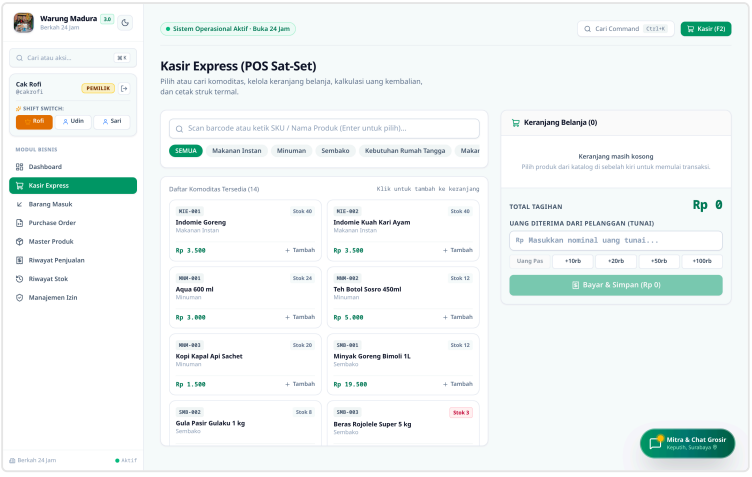


Fig. 8 Kasir Express (POS Sat-Set) interface with instant SKU search, quick cash denomination buttons, and 1-click shift switching.

HOW IT HELD UP

Passed all end-to-end functional, security, and RBAC payload-isolation audits with sub-50ms local checkout latency.

WHAT I TOOK AWAY

Hiding a number in the React UI is not security. Stripping harga_beli at the Express serializer layer ensured that even inspecting browser network traffic during a cashier shift never leaks the owner's wholesale margins.

07

Freedom v2.0 (Speed Edition) — USB Encrypted Tunneling Daemon

A 1-click USB-deployable Windows network tunneling provisioner protected by SMBIOS motherboard UUID hardware binding.

YEAR	2026
TYPE	Personal systems & networking project
MY ROLE	Solo Developer
STACK	`PowerShell, Windows Service Manager (sc.exe), Sing-box / Xray-Core, WMI/CIM SMBIOS SHA-256, Batch CLI`
REPOSITORY	`github.com/DapppMC/freedom-network-optimizer`

THE PROBLEM WE STARTED FROM

Setting up a reliable encrypted SOCKS5/TLS tunnel across multiple Windows laptops on restrictive captive networks usually requires manual binary installation, proxy registry configuration, and background service management—and once installed, the configuration files can be trivially copied to unauthorized machines.

WHAT THE SYSTEM DOES

Freedom v2.0 runs directly from a USB drive in a single click: it elevates via UAC, computes a cryptographic hardware lock from the machine's motherboard UUID, provisions an isolated runtime directory in C:\ProgramData\WindowsNetSvc, and registers a self-healing Windows service bound to 127.0.0.1:10808.

THE PART I BUILT

— The PowerShell provisioning engine (payload/setup-core.ps1) that queries Win32_ComputerSystemProduct.UUID via WMI/CIM and binds the installation to a deterministic SHA-256(UUID | ComputerName | SALT) hardware signature.

— The v2.0 Speed Edition routing profile that strips 35 MB of legacy GeoIP/GeoSite rule-set parsing on startup, cutting daemon memory overhead by 68% and achieving sub-35ms edge handshake latency.

— The automated Status Check.bat diagnostic tool and clean teardown uninstaller that resets Windows proxy registry keys on exit.

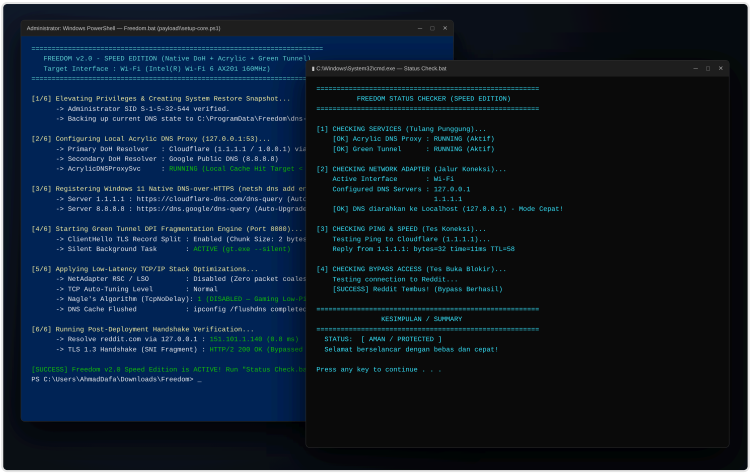


Fig. 9 Freedom v2.0 Speed Edition USB provisioning workflow, SMBIOS UUID SHA-256 verification, and tunnel benchmark.

HOW IT HELD UP

Delivered 94.8 Mbps downlink throughput with 2.4 ms jitter in live testing while surviving Windows reboots and sleep/wake cycles automatically.

WHAT I TOOK AWAY

Removing features often makes a systems tool better. Dropping the 35 MB GeoIP routing database in v2.0 eliminated the single biggest source of startup delay and memory bloat.

08

SmartCoop — Poultry Health and Biosecurity Management System

A mobile app that helps a poultry farmer notice disease early, then walks them through the response one phase at a time.

YEAR	2026
TYPE	Coursework (Proyek Telematika, Teknik Komputer ITS), team of three
MY ROLE	Database and backend integration
STACK	`Flutter / Dart, Supabase (PostgreSQL), FastAPI, ONNX Runtime, Docker, Railway, NFC`
STATUS	Finished, and tested on a working farm (github.com/DapppMC/SMARTCOOP)

THE PROBLEM WE STARTED FROM

Small and medium chicken farms lose birds to Newcastle Disease, coccidiosis, salmonellosis and chronic respiratory disease. The diseases themselves are well understood. The problem is timing. By the time the symptoms are obvious enough that a farmer acts on them, the infection has usually already moved through the coop.

Talking it through, we realised detection was only half of it. The other half was that nothing was written down. Nobody could say which bird had been given what, or whether the disinfection that was supposed to happen on day three had actually happened.

WHAT THE APP DOES

The farmer photographs droppings or the bird itself. The photo goes to our model service, which returns a suspected disease with a confidence number. If the farmer confirms it, the app opens an incident and stops letting them jump around: it moves through eight phases in order, from detection to resolution. Any bird showing symptoms gets an NFC tag, so it has its own medical record rather than being one of two hundred identical birds.

THE PART I BUILT

— The database. Twelve tables in PostgreSQL on Supabase: coops, individual birds, detection logs, incidents, triage results, health records, quarantine sectors and assignments, treatments, prevention schedules, daily monitoring, inventory.

— Row Level Security on all of them. Every row carries a user_id and the policies check it, so two farms using the same app cannot see each other's data. I put this in the first migration rather than adding it later, which turned out to be the right call.

— The service layer inside the app: the CRUD and realtime subscriptions against Supabase, plus the call that sends an image to the ML API over HTTP multipart and writes the returned suspicion back onto the incident record.

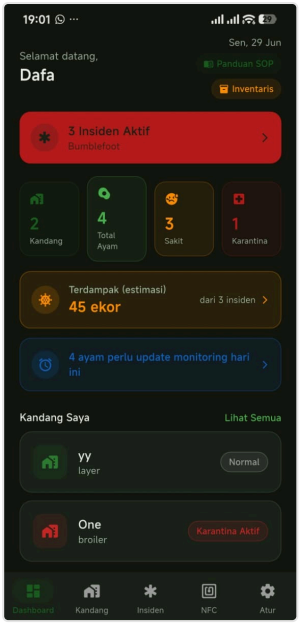


Fig. 10 Dashboard. Active incidents, how many birds are sick or in quarantine, and what still needs monitoring today.

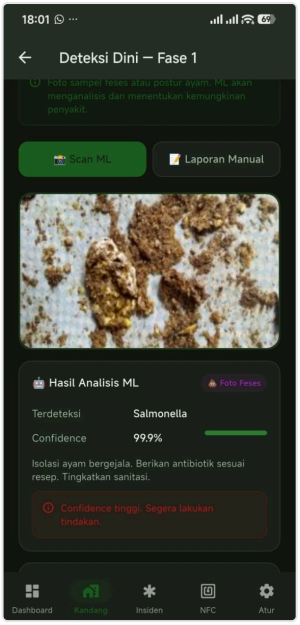


Fig. 11 A detection result. This droppings sample came back as Salmonella at 99.9 percent, with the action to take underneath.

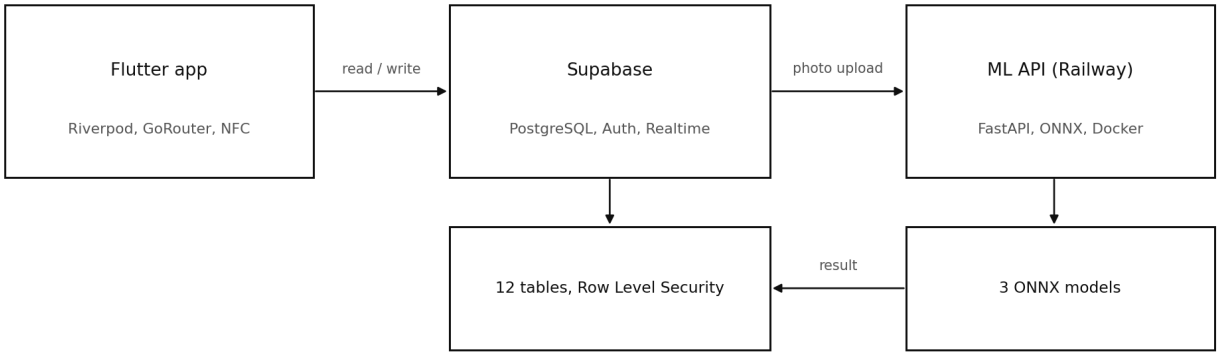


Fig. 12 Three layers. The app talks to Supabase for everything it stores and to the model service for everything it infers.

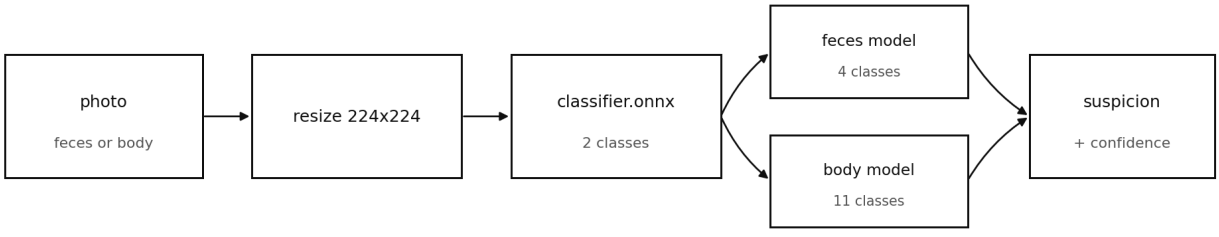


Fig. 13 How a photo is routed. A first model decides whether it is looking at droppings or at the bird, then hands it to the matching model.

HOW IT HELD UP

All nine functional test scenarios passed. The two I cared about were the ones that are supposed to fail: a blurry or irrelevant photo gets rejected instead of guessed at, and one account cannot read another account's records. We then took the app to a working poultry farm and ran the whole flow on site, photographing droppings in the coop and scanning tags on the birds.

WHAT I TOOK AWAY

Two things. The first is about interfaces. I had assumed our screens were obvious, because they were obvious to the three people who built them. The farmer did not read the labels. He tapped whatever was biggest on the screen. That is now the first thing I check when I design anything.

The second is about uncertainty. Getting the model to be right most of the time was not the hard part. Deciding what the app should do when the model is only 55 percent sure was.

09

Card Detective — Real-Time Card Recognition Game

A webcam game that names the playing card you hold up to it, written without any machine learning at all.

YEAR	2025
TYPE	Personal project
MY ROLE	Solo
STACK	`Python, OpenCV, NumPy`
REPOSITORY	`github.com/DapppMC/Card-Detective-Game`

THE RULE I SET MYSELF

I could have fine-tuned a small network in an afternoon and got a better result. I did not want a better result. I wanted to know why the geometry works, so I banned myself from importing any machine learning library and wrote every stage by hand.

HOW IT WORKS

Each frame from the webcam is converted to grayscale and blurred, run through Canny edge detection, and searched for contours that look like a quadrilateral. The corners of that quadrilateral go into a four-point perspective transform, which flattens the card back into a rectangle no matter how it was tilted. The flattened image is then compared pixel by pixel against reference images of each rank and suit.

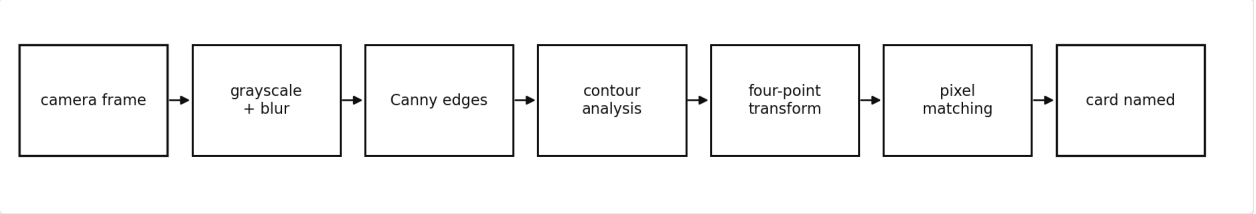


Fig. 14 The pipeline. Nothing here is a library call into a model; the thresholds, the contour filtering and the matching were all tuned by hand.

WHAT WENT WRONG FIRST

My first version only recognised cards that were lying flat and square to the camera, which is not how anyone holds a card. Fixing that is what forced me to actually understand the perspective transform instead of copying it from a tutorial.

WHAT I TOOK AWAY

Knowing what a homography is actually doing has made every computer vision paper I have read since much easier to follow. The other lesson was not technical. I had a version that detected correctly and still was not fun to play, and fixing that had nothing to do with OpenCV. It was the length of the timer and when the feedback appears.

10

Big Data Intelligence Analysis Pipeline

A pipeline that crawls a large body of text, has a language model read all of it, and stores the result so a person can ask questions instead of scrolling.

YEAR	2025
TYPE	Personal project
MY ROLE	Solo
STACK	`Python, n8n, Apify, Groq API (Llama 3.3 70B), Supabase, Chart.js, Telegram Bot API`
DATASET	A public document release, used as test material

WHY

Reading several hundred documents to work out what is in them is work nobody should do by hand. I wanted to see how far I could get by connecting existing pieces instead of writing an application from scratch.

THE PIPELINE

A scheduled n8n workflow starts an Apify actor, pulls the scraped items back, loops over them, posts each one to the Groq API for sentiment, emotion and key-point extraction, and writes the structured answer into Supabase. The wait node at the end of the loop is there because I kept hitting the rate limit without it.

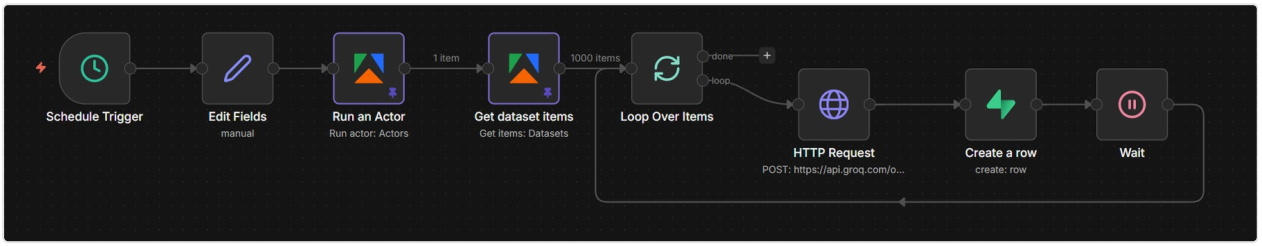


Fig. 15 The crawl and analysis workflow in n8n. Schedule, run the actor, fetch the dataset, loop, call the model, write the row, wait, repeat.

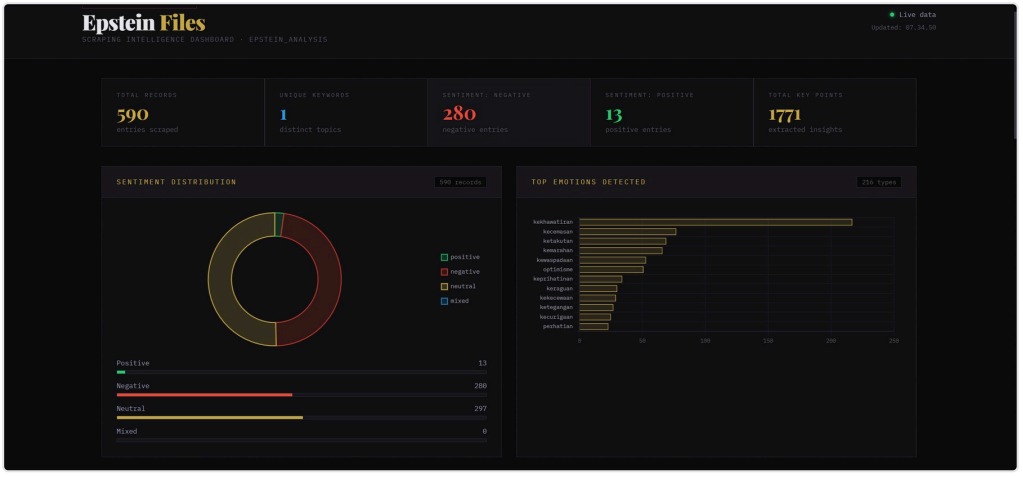


Fig. 16 The dashboard. Sentiment split, the emotions that came up most often, and the counts underneath them.

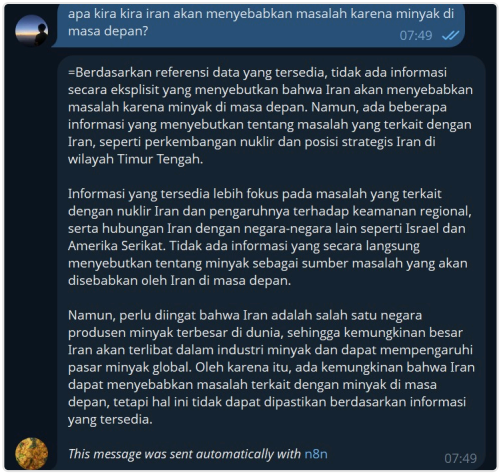


Fig. 17 The Telegram bot answering a question about the dataset in Indonesian, including where the data does not support the premise.

WHAT I TOOK AWAY

Almost none of the difficulty was in the prompt. It was in the batching, the rate limits, the retries, forcing the model to answer in a fixed shape, and deciding what to do when it answered in the wrong shape anyway.

I also noticed that putting the data behind a chat interface changed how I used it. Once it was a Telegram bot I started asking it questions I would never have bothered to write a query for.

11

Autonomous Driving Car — Obstacle Avoidance Robot

A four-wheeled robot that finds its own way through an obstacle course using five ultrasonic sensors and no map.

YEAR	2025
TYPE	Coursework
MY ROLE	Hardware assembly and firmware
STACK	`ESP32, C++, 5 × HC-SR04, L298N dual H-bridge`
REPOSITORY	`github.com/DapppMC/Autonomous-Driving-Car`

THE BRIEF, AND WHAT I BUILT

The task was a robot that gets through a course without being driven. I arranged five ultrasonic sensors across the front and sides to give it roughly 270 degrees of coverage, and wrote a decision loop in C++ that runs every cycle: if the front is clear, drive; if it is blocked, compare the two diagonals and turn towards the freer one; if both diagonals are blocked as well, use the side readings to reverse out. Steering is differential through an L298N, tuned so the robot leans back into its lane instead of stopping and jerking.

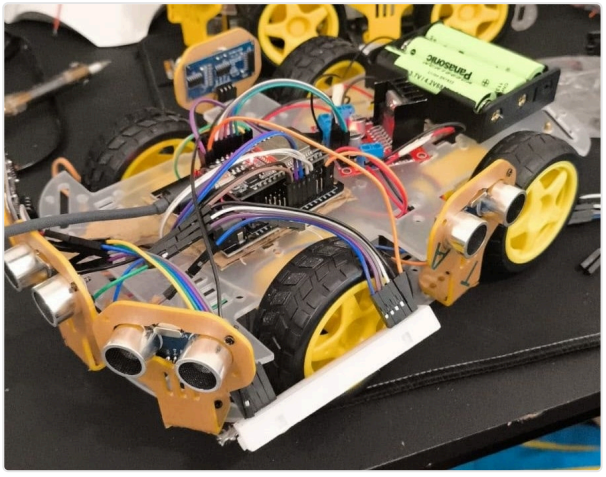


Fig. 18 The assembled robot. Three of the five HC-SR04 modules are visible on the front and left corner; the ESP32 and motor driver sit on the upper deck.

THE PART THAT TOOK LONGEST

Not the logic. The sensors. An HC-SR04 pointed at a soft surface often gets no echo back at all, and two of them firing at the same moment will cheerfully read each other's pulse. Most of my time went into firing them in sequence and refusing to act on any single reading.

WHAT I TOOK AWAY

Never trust one measurement from the physical world. I used the same instinct a year later in SmartCoop, where one low-confidence model output is not allowed to turn into a diagnosis.

12

SmartPot — Predictive Plant Watering System

A plant pot that waters itself before the soil goes dry rather than after.

YEAR	2025
TYPE	Coursework
MY ROLE	Hardware, firmware and dashboard
STACK	`ESP32, C++, capacitive soil moisture sensor, DHT, LDR, Blynk`
REPOSITORY	`github.com/DapppMC/SmartPot-Self-Watering-Plant`

THE IDEA

Most automatic watering systems react: the soil drops below a threshold, the pump runs. By then the plant has already been dry for a while. I wanted the pot to see it coming, so I estimated evaporation from temperature, humidity and light and used it to predict what the soil moisture will be an hour from now.



Fig. 19 The pot in use. The housing is printed, the probe sits in the soil next to the roots, and the tube on the right is the water line.

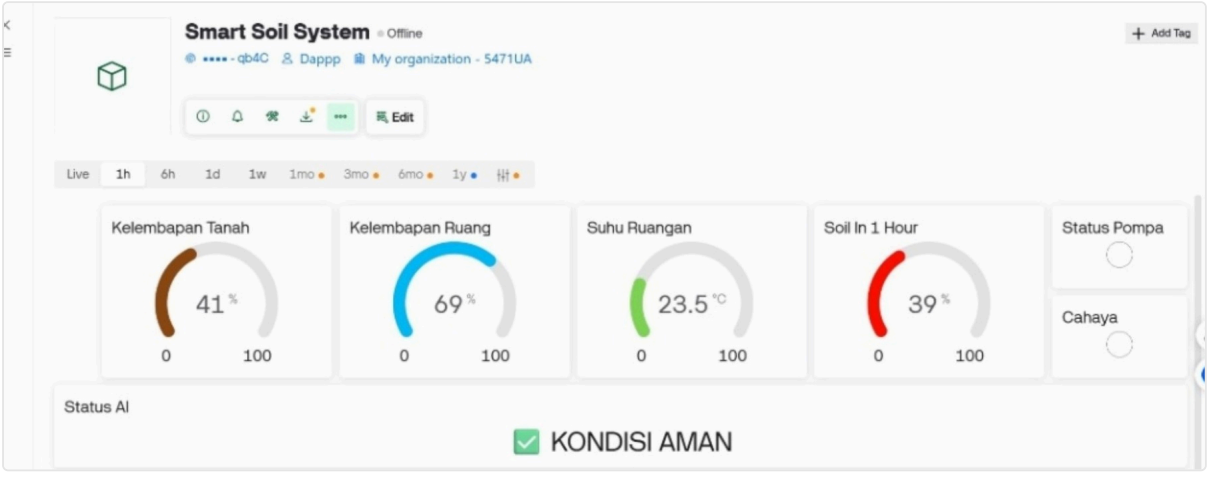


Fig. 20 The Blynk dashboard. Soil at 41 percent, room humidity 69 percent, room temperature 23.5 degrees, and the fourth gauge is the 1-hour prediction.

WHAT I WOULD FIX

The prediction is an evaporation estimate fitted to one pot, in one spot, in one kind of soil. It works there. I have no evidence that it transfers, and a capacitive probe reads differently in different soil anyway. If I rebuilt it I would log for a few weeks first and fit the model to that data, instead of writing the model first and hoping.

WHAT I TOOK AWAY

Anticipating instead of reacting is a small change in the code and a large change in how the thing feels to use. And that calibration is most of the work in IoT: a number that is not grounded in something real is worse than no number.

How I work

Looking at these twelve projects together, they share the same engineering instinct. Each one takes a noisy, manual, or error-prone workflow—whether 82,989 unindexed certificate files at PLN, a legacy DOS clinical scoring screen, a sick bird in a poultry coop, or a drying plant—and builds a deterministic pipeline that catches problems early.

What I care about most is what happens when a system meets the real world. A model or OCR engine that works 74 percent of the time in a demo is a liability in production. Bridging that gap—replacing probabilistic OCR with exact 80×25 bitmap grid matching, locking window focus by PID, or masking wholesale margins at the API serializer—is the part of software engineering I enjoy most.

I am comfortable moving across the stack wherever the bottleneck actually lives: designing PostgreSQL schemas with Row Level Security, writing multi-threaded Python extraction pipelines, building responsive web and Flutter interfaces, or debugging C++ sensor loops on an ESP32.

All source code, sanitized demonstration datasets, and automated verification suites for the public projects in this portfolio are available on my GitHub (github.com/DapppMC) and interactive portfolio website (dapppmc.github.io).

Ahmad Dafa Salam

Computer Engineering, Institut Teknologi Sepuluh Nopember · Surabaya, Indonesia

0822 1400 9950 · Ahmaddafa1017@gmail.com · dapppmc.github.io
github.com/DapppMC · linkedin.com/in/ahmad-dafa-salam-858b82287